CLOUD COMPUTING

Assignment No-3 (OWN CLOUD)

ORBIT CLOUD

Your files. Your cloud. Anywhere.

Student Name	Ijaz Ahmad
Reg No.	83500
Enrolment #	03-134231-021
Group	3
Instructor	Dr Nadeem Sarwar
Date	6 November 2026

Question 1 · Tool selection | Question 2 · Own cloud | Question 3 · Diagram

Summary

This assignment builds a working private cloud named **ORBIT CLOUD**. Clients store files in it and can **ADD, UPDATE, DOWNLOAD and DELETE** them from home, from the office or from a phone, through a web browser, a desktop/mobile app or any WebDAV client, over encrypted HTTPS.

Question	Answer in one line
Q1 Tool	**Nextcloud 34.0.5** (open-source private-cloud platform) running on **Docker Compose**, chosen after comparing seven tools.
Q2 Own cloud	**ORBIT CLOUD**: gateway + application + database + cache + background worker, a provider account and two client accounts (home, office) with storage quotas, version history, trash bin and backups.
Q3 Diagram	An architecture diagram (Figure A) and a request-lifecycle diagram (Figure B) show how a request travels through the cloud.
Evidence	31 checks passed, 0 failed in the automated test; backup and restore proven by recovering a permanently deleted file.

Question 1 — Tool to create the cloud

1.1 What the tool must do

Question 2 asks for a cloud that the owner can reach from home or the office and use to download, delete, add and update files, offering services that cloud providers give to clients. So the tool must:

- store **real files** for **separate user accounts** (a provider and clients), not only simulate load;
- be reachable over a **network** through a web interface and standard protocols;
- run on **one ordinary computer**, quick to deploy, with **our own name and branding** (the “TIGER CLOUD” idea of the question).

1.2 Tools compared

Tool	What it is	Fit for this assignment	Verdict
Nextcloud	Open-source, self-hosted file storage, sync and sharing platform (web, desktop, mobile, WebDAV).	Real uploads/downloads, accounts, quotas, versions, trash, sharing, theming. Runs on one machine.	**Selected**
ownCloud	Older project from which Nextcloud was forked (2016); same idea.	Would work, but a smaller feature and app ecosystem.	Alternative
Seafile	File sync and share server focused on speed.	Good for files; fewer built-in cloud-style management features.	Alternative
OpenStack	Full IaaS cloud (compute, network, storage, identity) — studied in Assignment 1.	Needs several servers/VMs and a lot of RAM; far more than one laptop can host comfortably.	Too heavy
CloudSim	Java simulator of data centres, VMs and schedulers.	Simulates behaviour only — no real files, no users, no remote access.	Cannot meet Q2
LocalStack	Local emulator of AWS APIs (S3, etc.) for developers.	API-only, no end-user portal; imitates AWS rather than being our own named cloud.	Not suitable
MinIO	S3-compatible object storage server.	Strong storage engine, but a building block, not a ready cloud for end users.	Not suitable

1.3 Decision

Nextcloud 34.0.5 was selected because it is the only option that satisfies every requirement at once: real file operations, accounts and quotas, a web portal plus mobile/desktop apps, custom branding, and light enough to run on one

computer. It is also widely used in practice, so the skills transfer.

1.4 Supporting tools

Tool	Why it is used
Docker + Docker Compose	Virtualisation at operating-system level. The whole cloud is described in one file (`docker-compose.yml`) and can be built, stopped, moved or rebuilt with one command.
PostgreSQL 17	Database service holding users, file index, shares and quotas (more robust than the default SQLite).
Redis 7	Cache and file-locking service so two clients cannot corrupt the same file.
Caddy 2	Gateway: terminates HTTPS with a certificate from a private certificate authority and forwards requests to the application.

Question 2 — Our own cloud: ORBIT CLOUD

2.1 Identity of the cloud

The cloud carries its own name, slogan, colour and logo (theme settings applied automatically at start-up): **ORBIT CLOUD** — “Your files. Your cloud. Anywhere.”. The name is a single setting (`CLOUD_NAME` in `.env`), so it can be changed to the group’s own name at any time.

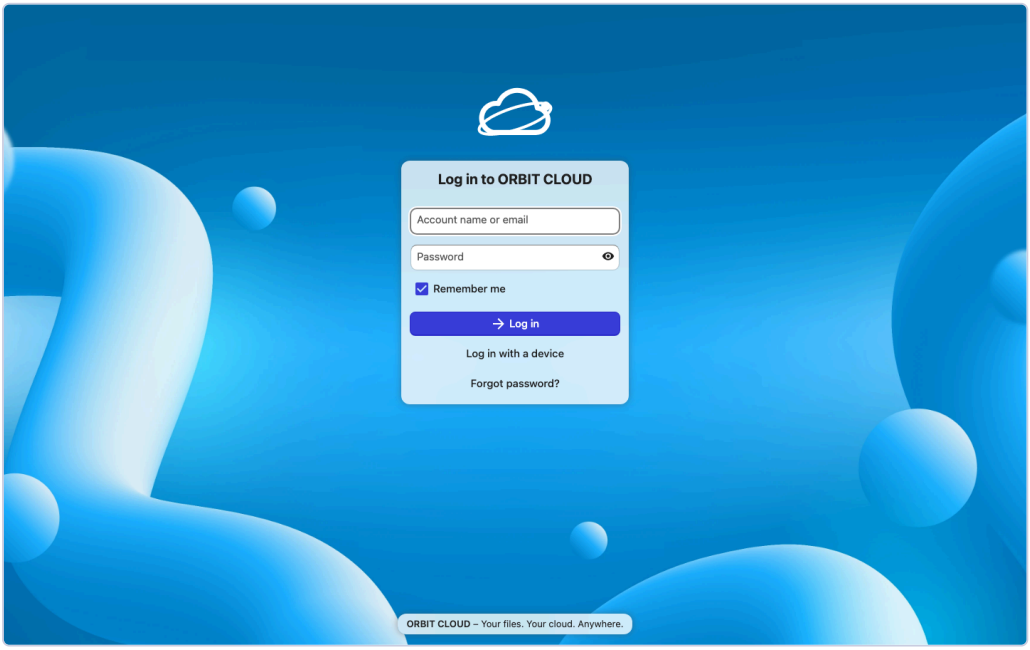


Figure 1 — ORBIT CLOUD login page with its own name, logo and slogan.

The cloud has the two roles the question describes — a **provider** and **clients**:

Role	Account	Purpose
Provider (administrator)	orbitadmin	Runs the cloud: creates clients, sets quotas, monitors services.
Client — home	home	Uses the cloud from home. Storage quota: **5 GB**.
Client — office	office	Uses the cloud from the office. Storage quota: **10 GB**.

Passwords are generated randomly during set-up and kept only in a private `.env` file (permissions 600); they are not written in this report or in the source code.

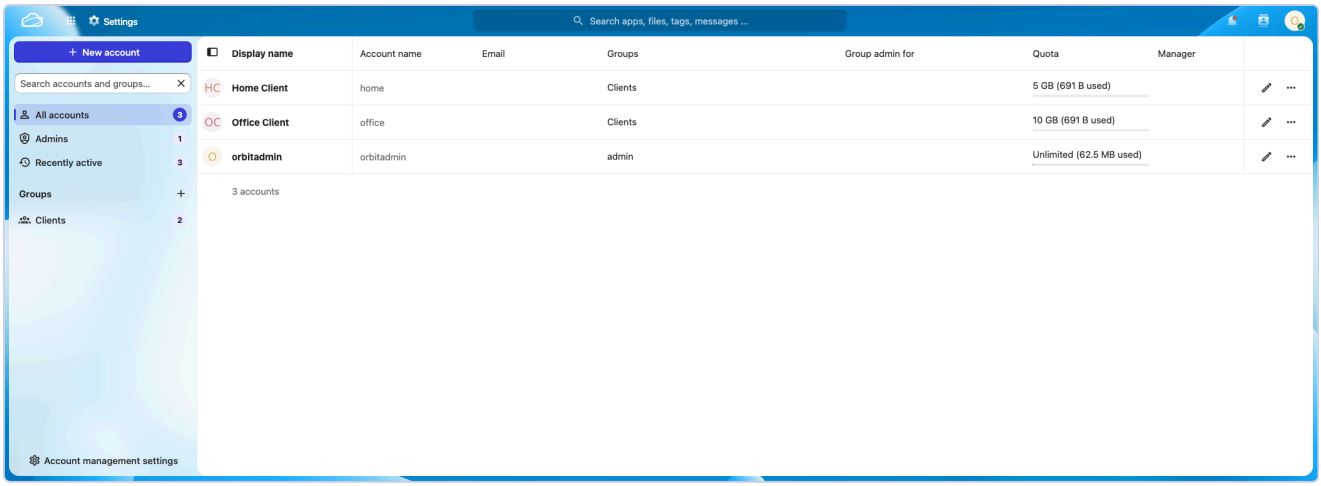


Figure 2 — Provider view: the administrator sees every client account with its storage quota.

2.2 Services provided and how they relate to Assignment 1

Assignment 1 listed the building blocks of a cloud platform (OpenStack components, Docker and Compose, web/proxy servers, databases, SSL/TLS, quotas, backup tools). The cloud below is built from the same kinds of services, scaled to one machine:

Service in the cloud	Technology	Model	Comparable service from Assignment 1
File storage & sharing (the main service)	Nextcloud Files	SaaS	OpenStack Swift (object storage) and Manila (shared file system)
Web portal	Nextcloud web UI	SaaS	OpenStack Horizon (dashboard)
Identity & access (accounts, groups, passwords)	Nextcloud users	—	OpenStack Keystone (user management)
Metering & quotas	Quota + usage report	—	OpenStack Ceilometer (usage measurement); disk quota
Database service	PostgreSQL 17	DBaaS	OpenStack Trove; PostgreSQL server
Cache / locking service	Redis 7	—	Redis server
Virtual network	Docker network `orbit-net`	IaaS	OpenStack Neutron; Docker network basics
Reverse proxy + TLS	Caddy 2	—	OpenStack Octavia; Nginx reverse proxy; self-signed SSL/TLS
Compute (virtualisation)	Docker containers	IaaS	OpenStack Nova; Docker
Block storage	Docker volumes	IaaS	OpenStack Cinder; external storage for containers
Images	Docker images	IaaS	OpenStack Glance
Orchestration	Docker Compose	—	OpenStack Heat; Docker Compose
Backup & recovery	`./orbit backup` / `restore`	—	Cinder Backup; rsync

> The comparison is by function: ORBIT CLOUD is not OpenStack, but a small private cloud that delivers the same kinds of services from one computer.

2.3 How it was built

Everything is automated by one control script (`./orbit`) so the cloud can be rebuilt on any computer with Docker:

```
./orbit setup        # create .env with random passwords
./orbit up           # download, install, brand, create clients and quotas
./orbit credentials  # show the logins
./orbit test         # prove ADD / UPDATE / DOWNLOAD / DELETE work
```

What `./orbit up` does, step by step:

1. Detects the computer's network address and lists every name the cloud will answer to.
2. Starts five containers on a private network: **gateway**, **app**, **db**, **cache**, **cron**. Only the gateway publishes a port (8443); the database and cache are unreachable from outside.
3. Waits until the database, cache and application report healthy.
4. Applies the branding, enables background jobs, and trusts the gateway as a proxy.
5. Removes the product-tour pop-up and installs ORBIT CLOUD's own welcome files for new clients.
6. Creates the group "Clients", the two client accounts and their quotas.
7. Exports the HTTPS root certificate so devices can trust the cloud.

2.4 Access from home or office

The cloud listens on HTTPS port 8443 of the server computer. Any device that can reach the server opens:

From	Address
The server itself	`https://localhost:8443`
Home or office (same Wi-Fi / LAN)	`https://192.168.100.181:8443`
Anywhere on the Internet	Through a private VPN or tunnel (e.g. Tailscale): add the VPN name to `ORBIT_EXTRA_HOSTS`, run `./orbit up`, open `https://<vpn-name>:8443`.
Android phone (e.g. Google Pixel)	Same Wi-Fi: open the address in Chrome, accept the one-time certificate notice, log in. Optionally install the Nextcloud app from Google Play and enter the same address.

The browser shows a certificate notice the first time, because the certificate comes from the cloud's own certificate authority. `./orbit trust-ca` exports that certificate, and `./orbit share-ca` lets a phone download it over Wi-Fi, so each device can trust it once.

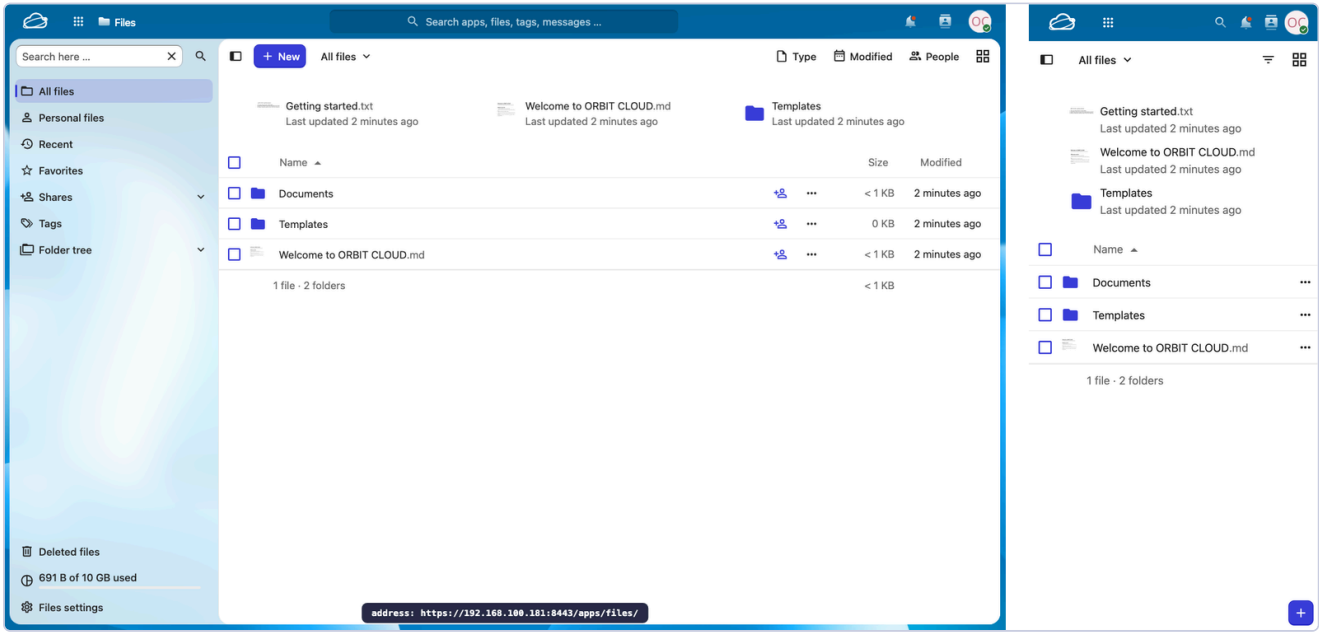


Figure 3 — Left: client "office" signed in through the network address (192.168.100.181), the way an office device reaches the cloud. Right: the same cloud in a phone-sized window (mobile layout, emulated in the browser).

> Scope of testing: access was verified through `localhost` and through the computer's LAN address on this machine. A second physical device on the same network uses the identical LAN address; Internet access additionally needs the VPN/tunnel step above, which was documented but not exercised here.

2.5 File operations: ADD, UPDATE, DOWNLOAD, DELETE

The four operations the question asks for were performed in two independent ways: through the **web interface** (screenshots below: ADD and DELETE were done by clicking in the browser) and through an **automated test** that talks to

the cloud exactly as the desktop and mobile apps do (WebDAV over HTTPS, certificate verified).

A. ADD — upload a file

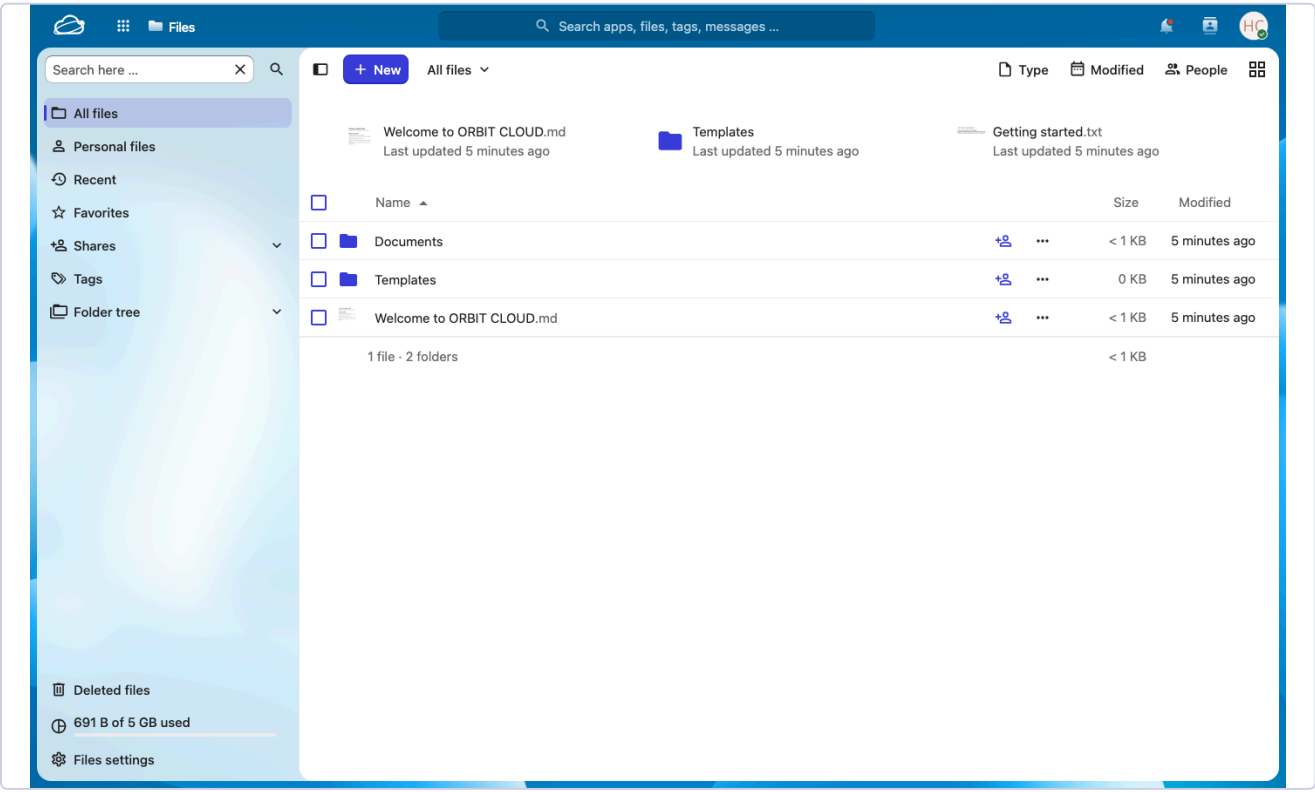


Figure 4 — Client "home" signed in: the personal file area.

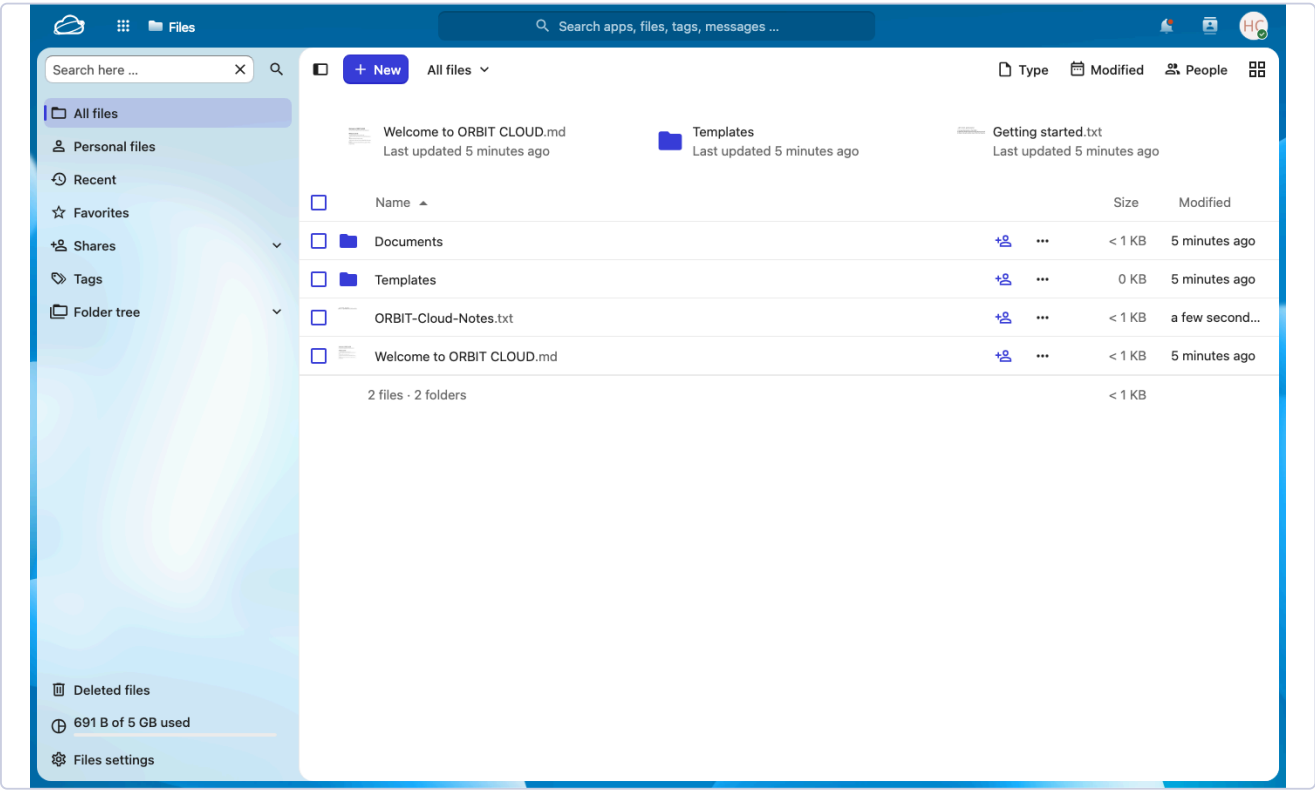


Figure 5 — ADD: a new file uploaded through the web interface appears in the list immediately.

B. UPDATE — change a file (old version is kept)

Uploading new content under the same name replaces the file, and the cloud keeps the earlier copy in the version history so a mistake can be undone. Here the update was sent with the same WebDAV request the desktop and mobile apps use, and the web interface then shows the history.

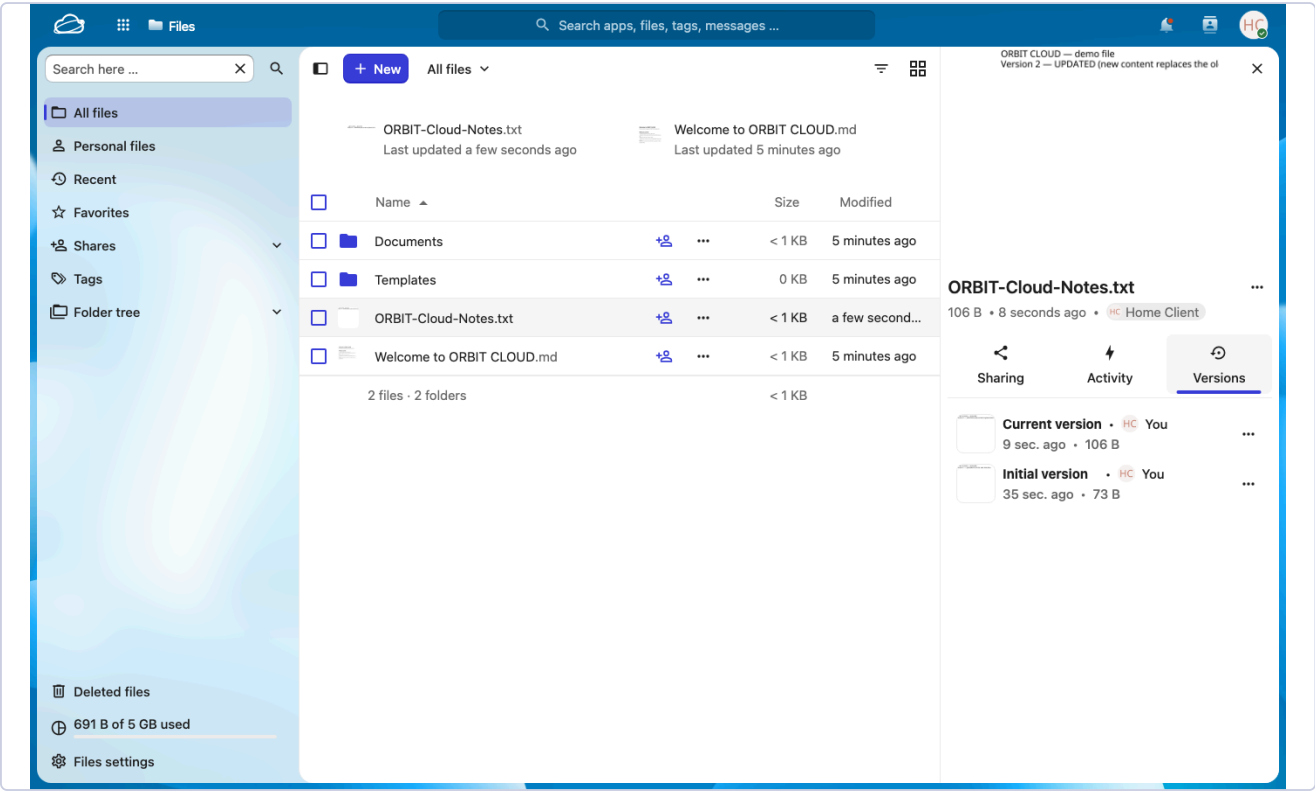


Figure 6 — UPDATE: version history after the second upload — "Current version" (106 B, the new content) and "Initial version" (73 B, the original, still stored).

C. DOWNLOAD — fetch a file

The automated test downloads each file and compares its SHA-256 checksum with the original; they were byte-identical (see Appendix B).

D. DELETE — remove a file (recoverable)

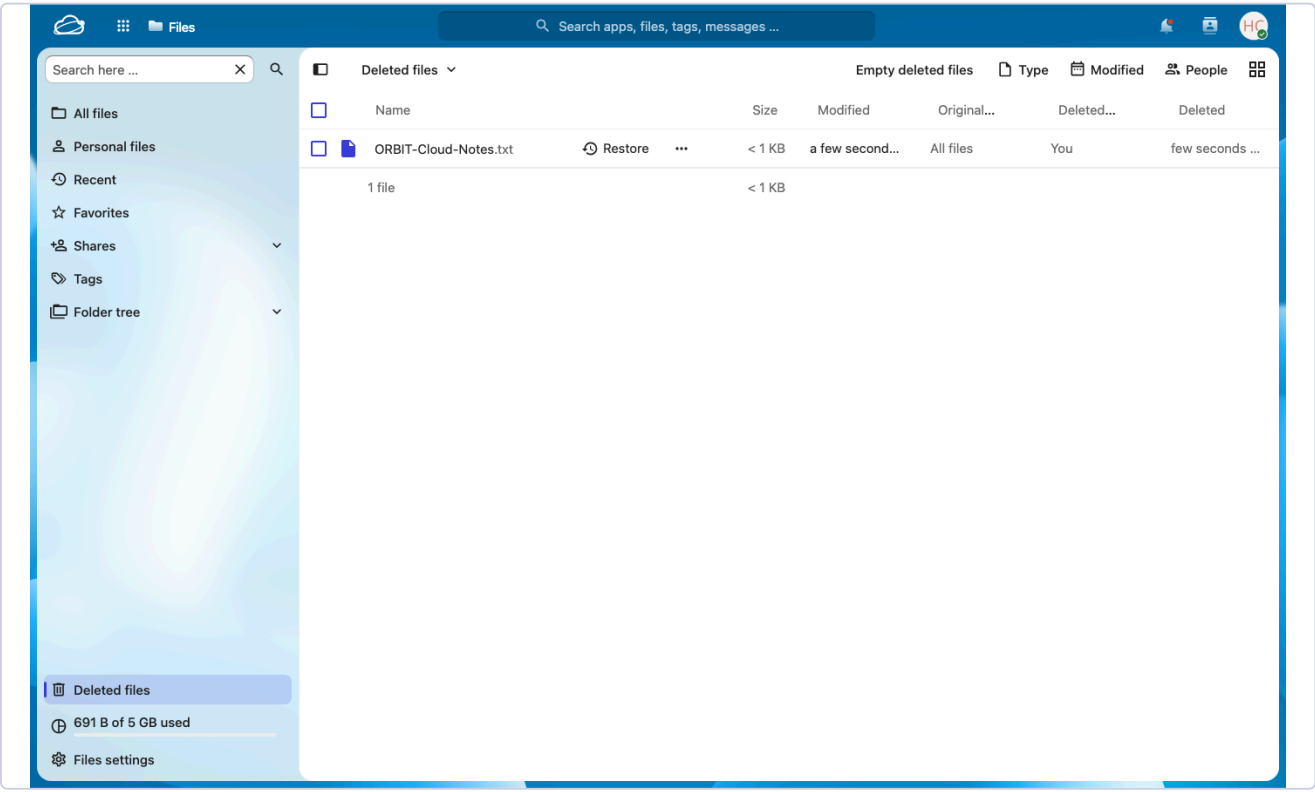


Figure 7 — DELETE: a removed file moves to "Deleted files" and can be restored until the trash expires.

2.6 Automated verification

The test `./orbit test` acts as two clients: "home" connects through `localhost`, and "office" connects through the LAN address like a second device. Result: **31 checks passed, 0 failed**

Test	What was proved
1	Security: requests without credentials or with a wrong password are rejected (HTTP 401); valid credentials are accepted.
2	ADD: a folder and a file are created and listed.
3	DOWNLOAD: the file comes back byte-identical (SHA-256 match).
3b	Isolation: the Office client cannot read, write, delete or list the Home client's files (HTTP 404 / 404 / 404 / not listed); Home's file is untouched afterwards.
4	UPDATE: the new content is served, and the earlier version is stored and downloads byte-identical to the original upload.
5	Large file: a 25 MB upload and download are byte-identical.
6	SHARE: Home shares a file read-only with Office; Office downloads it from the LAN address, cannot modify it (HTTP 403), and loses access when the share is removed.
7	DELETE: the file disappears (HTTP 404), appears in the trash bin, and is restored with the latest content.
8	METERING: each client has a storage quota; used and available space are reported.
9	Clean-up: test data and trash are removed.

2.7 Security and reliability features

Feature	How it is provided
Encryption in transit	All traffic uses HTTPS; the test connects with TLS 1.3 and verifies the certificate chain.
Authentication	Every request needs a valid login; wrong or missing credentials get HTTP 401.

Feature	How it is provided
Isolation between clients	Each client sees only their own files; sharing is explicit and permission-controlled. Tested: another client's attempts to read, write, delete or list files all fail.
Reduced attack surface	Only port 8443 is published; the database and cache live on the private `orbit-net` network.
Secrets handling	Random passwords generated per installation, stored in `.env` with mode 600, never in code.
Mistake recovery	Version history for updated files and a trash bin for deleted files.
Consistency	Redis file locking and database transactions prevent two clients from corrupting one file.
Backup & restore	`./orbit backup` pauses writes, dumps the database, archives configuration and user files and writes SHA-256 checksums to `backups/<date-time>/`; `./orbit restore` verifies the checksums first. Tested: a file was permanently deleted (trash emptied), the backup was restored, and the file returned byte-identical.
Metering	Per-client quotas (5 GB / 10 GB) and usage reports (`./orbit status`).

2.8 Limitations and possible improvements

- **Single server:** if the host computer is off, the cloud is unavailable (a cluster with replicated storage would remove this single point of failure).
- **Private certificate:** devices must trust the cloud's certificate authority once; a public domain with a public certificate would remove the notice.
- **Internet access** needs a VPN or tunnel (opening router ports to the Internet is deliberately avoided). **Production hardening** would add two-factor authentication, HSTS, off-site backups and regular image updates.

Question 3 — Diagram of the cloud and how it works

3.1 Architecture diagram

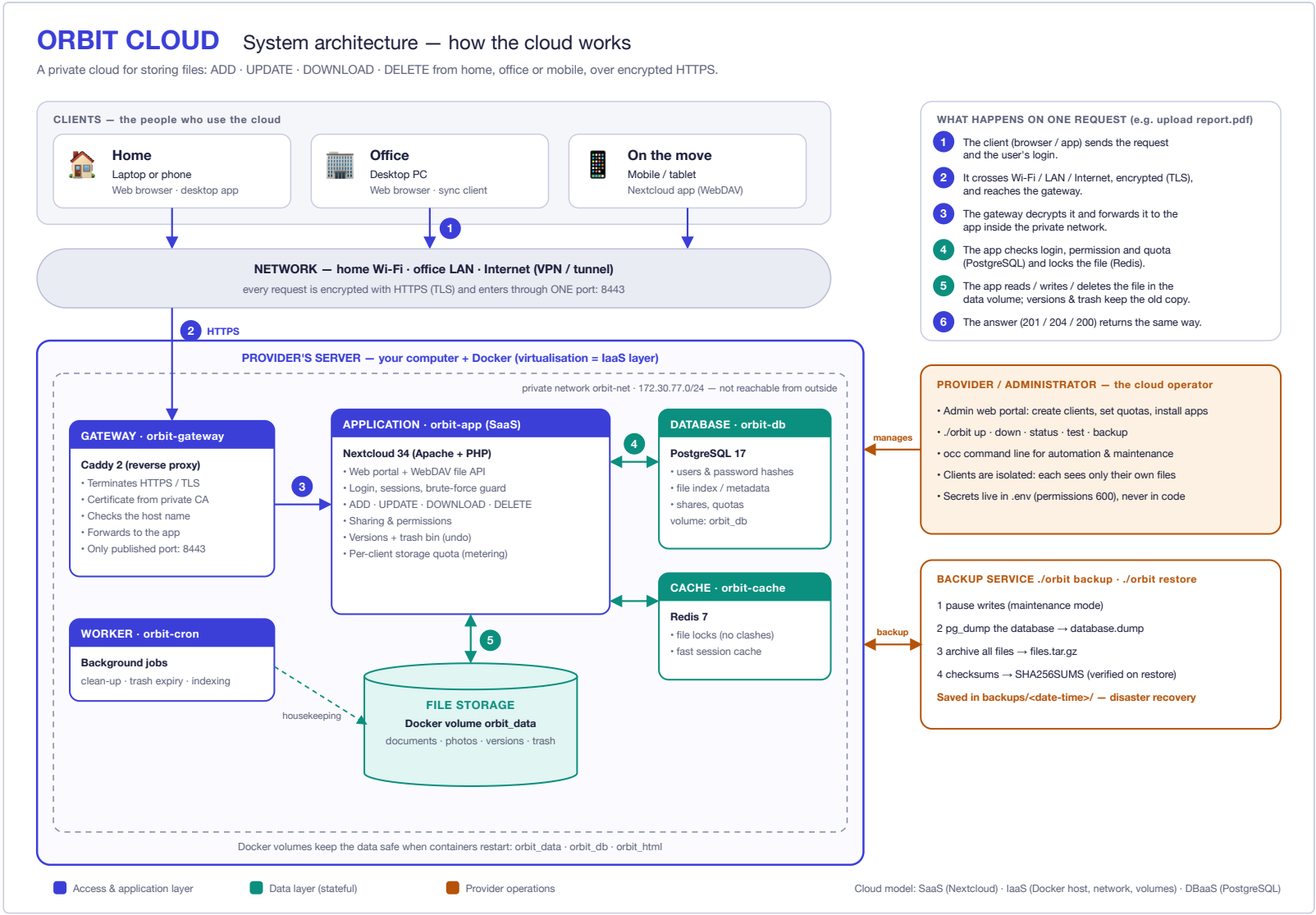


Figure A — Architecture of ORBIT CLOUD and the path of one request (numbers 1–6).

3.2 Request lifecycle diagram

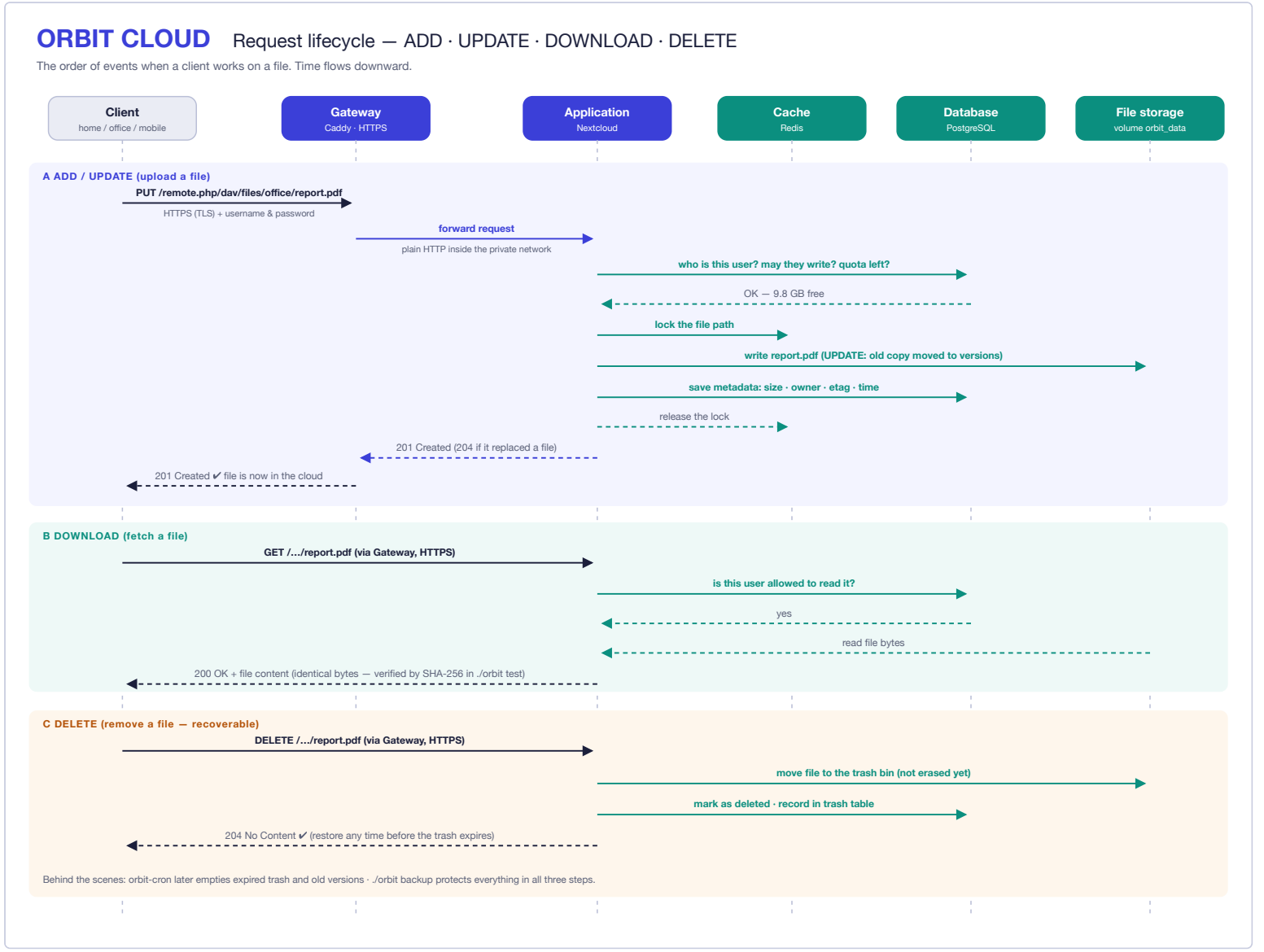


Figure B — Order of events for ADD / UPDATE, DOWNLOAD and DELETE.

3.3 Reading the architecture diagram

Step	What happens
1	A **client** (home laptop, office PC or phone) sends a request, for example “upload report.pdf”, together with the user’s login.
2	The request crosses the **network** (home Wi-Fi, office LAN or a VPN) **encrypted with HTTPS**, and enters the provider’s server through the single published port 8443.
3	The **gateway** (Caddy) decrypts it, checks the requested host name and forwards it to the application over the private network `orbit-net`.
4	The **application** (Nextcloud) checks the login, the permissions and the quota in **PostgreSQL**, and locks the file in **Redis**.
5	The application **adds, updates, reads or deletes** the file in the **data volume**; old versions and deleted files are kept for recovery, and the metadata is saved in PostgreSQL.
6	The result (HTTP 201 / 204 / 200) returns along the same path to the client.

Besides the request path, the **cron** worker does housekeeping (expired trash, old versions, indexing), the **administrator** manages the cloud through the web portal and the `./orbit` script, and the **backup service** protects the database and files.

3.4 Cloud characteristics demonstrated

Characteristic (NIST)	Where it is visible in this cloud
On-demand self-service	Clients log in and manage files themselves; the provider creates a new client with a single command.
Broad network access	Browser, desktop app, mobile app and WebDAV, from any network that can reach the server.
Resource pooling	One server, database and storage pool shared by several isolated clients (multi-tenancy).
Rapid elasticity	A client’s capacity changes instantly with one command (`occ user:setting <user> files quota "20 GB"`).
Measured service	Per-client quota and usage reporting (`./orbit status`), like metering in a public cloud.

Deployment model: private cloud (hosted on our own hardware for our own users). **Service models used:** SaaS (file-storage application) running on an IaaS-style container platform, with a database service (DBaaS) behind it.

3.5 Design trade-offs

Decision	Benefit	Cost
Private, self-hosted cloud	Full control and privacy; no subscription.	We maintain updates, backups and uptime.
Containers instead of virtual machines	Light, quick to start, one-file description.	Weaker isolation than full VMs.
Single host	Simple to build and explain.	Single point of failure (mitigated by backups).
HTTPS-only gateway with a private CA	Encrypted, one controlled entry point.	Certificate must be trusted once per device.
Nextcloud instead of custom code	Mature features: sharing, versions, apps.	Less freedom to change internals.

Appendix A — Project contents

```
Assignment3_ICC/
  Assignment3_ICC_Report.docx / .pdf   this report
  Q3_Diagrams/                         architecture + request-flow diagrams (PNG, SVG)
  Screenshots/                         screenshots used in the report
  Video/                               step-by-step video: opening the cloud on an Android phone
  VIVA_NOTES.md                        preparation notes for the viva
  docs/                                diagram sources, screenshots and test output
  README.md                            how to run the cloud
  orbit                                control script (setup, up, down, status, backup, restore, test)
  docker-compose.yml                   the five services, private network and volumes
  caddy/Caddyfile                      gateway configuration (HTTPS + reverse proxy)
  branding/                            logo and welcome files for new clients
  tests/file-ops.sh                    end-to-end ADD/UPDATE/DOWNLOAD/DELETE test
  tools/                               diagram and report generators
```

Appendix B — Output of `./orbit test`

Captured on 6 November 2026. A green tick marks each check that passed.

```
ORBIT CLOUD — end-to-end test   2026-11-06 15:12:59
  home   client → https://localhost:8443  (user: home)
  office client → https://192.168.100.181:8443  (user: office)
  transport     → SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF

1. Security — only authenticated clients get in
  ✔ No credentials is rejected  [HTTP 401]
  ✔ Wrong password is rejected  [HTTP 401]
  ✔ Home client logs in with valid credentials  [HTTP 207]

2. ADD — upload a folder and a file from Home
  ✔ Create folder 'orbit-e2e'  [HTTP 201]
  ✔ Upload hello.txt  [HTTP 201]
  ✔ File appears in the folder listing

3. DOWNLOAD — fetch it back and verify integrity
  ✔ Download hello.txt  [HTTP 200]
  ✔ Downloaded copy is byte-identical to the original  [sha256 b92222a72be8…]

3b. Isolation — one client cannot touch another client's files
  ✔ Office cannot READ Home's file  [HTTP 404]
  ✔ Office cannot WRITE into Home's folder  [HTTP 404]
  ✔ Office cannot DELETE Home's file  [HTTP 404]
  ✔ Office cannot LIST Home's files
  ✔ Home's file is untouched after the intrusion attempts  [HTTP 200]

4. UPDATE — overwrite the file, old version is kept
  ✔ Upload new content over hello.txt  [HTTP 204]
  ✔ Download returns the NEW content  [sha256 d4320b3b60a1…]
  ✔ Earlier version is kept: 1 stored version(s), and version 1 downloads byte-identical to the original upload

5. Large file — 25 MB round trip
  ✔ Upload 25 MB file  [HTTP 201]
  ✔ 25 MB download is byte-identical  [sha256 09f29568d9f6…]

6. SHARE — Home shares a file read-only with Office (Office connects from another device)
  ✔ Create share Home → Office (read-only)  [HTTP 200]
  ✔ Office client downloads the shared file via https://192.168.100.181:8443  [HTTP 200]
  ✔ Office copy matches Home's latest version  [sha256 d4320b3b60a1…]
  ✔ Office cannot modify a read-only share  [HTTP 403]
  ✔ After un-sharing, Office loses access  [HTTP 404]

7. DELETE — remove a file, then recover it from the trash bin
  ✔ Delete hello.txt  [HTTP 204]
  ✔ File is gone from the cloud  [HTTP 404]
  ✔ Deleted file is held in the trash bin (trash/hello.txt.d1793959992)
  ✔ Restore hello.txt from trash  [HTTP 201]
  ✔ Restored file has the latest content  [sha256 d4320b3b60a1…]

8. Metering — each client has a storage quota
  ✔ Home quota: 62 MB used, 5057 MB still available (limit 5 GB)

9. Cleanup
  ✔ Remove test folder  [HTTP 204]
  ✔ Empty trash  [HTTP 204]

Result: 31 passed, 0 failed
```